Planning Algorithms in AI

PS1: Discrete Planning

Gonzalo Ferrer
Skoltech
October 28th, 2026

This problem set has three tasks and is individual work. You are encouraged to talk at the conceptual level with other students, discuss on the equations and even on results, but you may not show/share/copy any non-trivial code.

Task	Topic	Points
1	Dijkstra and A* in 2D	30
2	State space (configuration space)	25
3	A* in 3D (position and orientation)	45
	Total	**100**

Submission Instructions

Your assignment must be submitted by 11:59pm on **Sunday, November 15**. You are to upload your assignment directly to Canvas as **two** attachments:

1. A `.tar.gz` or `.zip` file *containing a directory* named after your uniquename with the structure shown below.

   ```
   yourname_ps1.zip:
     yourname_ps1/
     yourname_ps1/any_code_file.py
     yourname_ps1/utils.py
     yourname_ps1/point_solve.gif
     yourname_ps1/rod_solve.gif
   ```

2. A PDF with the written portion of the problem set, reporting results, figures and answers to questions. Scanned versions of hand-written documents, note-books converted to PDFs, are perfectly acceptable (reduced size). No other formats (e.g., `.doc`) are acceptable. Your PDF file should adhere to the following naming convention: `yourname_ps1.pdf`.

Homework received after 11:59pm is considered late and will be penalized as per the course policy. The ultimate timestamp authority is the one assigned to your upload by Canvas. No exceptions to this policy will be made.

Task 1: Dijkstra and A* in 2D (30 points)

For this task, you will implement a graph search algorithm for a *point robot* (a single cell) in the 2D environment. This task is self-contained and does not depend on Tasks 2 or 3.

The environment map is provided in `data_ps1.npz` under the key `'environment'`: a 100×100 `np.ndarray` with value 0 for free cells and 1 for obstacles. It can be loaded with

```
data = np.load('data_ps1.npz')
environment = data['environment']
```

The point robot occupies a single cell, so it can plan directly on this map: a cell (x, y) is valid if it lies inside the grid and `environment[x, y] == 0`.

The state is $q = [x, y]$ and the actions allowed are moving up, down, left and right. In total 4 actions, each of them has an assigned cost of 1. The starting configuration of the agent is $q_S = (6, 6)$ and the goal configuration is $q_G = (85, 85)$.

A. **(20 pts)** Implement the Dijkstra and A* algorithms with the heuristic function $h(q, q_G) = 0$, and plan from start to goal. Report the number of visited states and the final cost. To visualize the state of the plan, you can use the function from `utils.py` `plot_env_2d(environment, state)`. To visualize the results of the plan you can use the function `plotting_results_2d(environment: np.ndarray, plan: list, save_path: str='point_solve.gif')` from `utils.py`, where `plan` is the list of states (x, y) from start to goal.

   hint: Track the number of visited states to avoid/debug potential issues with internal loops.

B. **(10 pts)** Change the heuristic function now to the L1 norm. Report the same metrics and compare them with Dijkstra in a table. How many states have been visited compared to Dijkstra? What is the final cost? Comment on the results.

Task 2: State Space (25 points)

For this task, you will write a Python script or note-book that generates the configuration space, including the obstacle regions, borders and free space. The Workspace is a rectangular grid area (image) of 100×100, such that $W = \{0, 1, \ldots, 99\} \times \{0, 1, \ldots, 99\} \subset \mathbb{Z}_0^+ \times \mathbb{Z}_0^+$.

The robot is a "rod" object, whose configuration is $q = [x, y, \theta]$, taking into account that x, y are the image coordinates, therefore they are integer values in the grid and the orientation θ is also discretized into several values.

All the required data can be obtained from `data_ps1.npz`. This numpy data file consists of the environment map `['environment'](np.ndarray([100,100]))`, an image with values 0 to indicate free space and 1 to indicate an obstacle. The second component is the robot/agent, whose shape is included in `['rod'](np.ndarray([11,11,4]))`. There are 4 dimensions to represent the different discretized orientations: the robot rotated by 0, 45, 90 and 135 degrees.

A. **(5 pts)** Visualize from the given data the workspace and the different rod configurations for each discretized orientation. Comment on the given discretized values for orientation: why are four values enough for this robot, and what is lost?

B. **(5 pts)** Visualize the environment together with the object. For this, you may want to use the function `plot_env_3d` from `utils.py` and select any valid configuration value for the rod.

C. **(10 pts)** Create the C-space for the 2D environment map. For this, plot all the images corresponding to each of the orientations by using collision checking.

   hint: you might want to look at the library `scipy.signal` and use function

   `signal.convolve2d(env_map, kernel, boundary='symm', mode='same')`

   to check for collisions.

   hint: you may want to use `normalize_image(img)` from `utils.py` to normalize the created space to $\{0, 1\}$, since after convolutions, values are not exactly 0 and 1 (this will be useful for task 3).

D. **(5 pts)** Comment on the obtained C-space with the previous method. What is the size of the C-space? How do the four orientation slices differ, and why?

Task 3: A* in 3D with Orientations (45 points)

For this task, you will extend your planner to the rod. The state is now $q = [x, y, \theta]$ and you will plan in the discrete C-space generated in Task 2. The actions allowed in this problem are moving up, down, left, right, rotate right and rotate left. In total 6 actions, each of them has an assigned cost of 1. The starting configuration of the agent is $(6, 6, 2)$ and the goal configuration is $(85, 85, 0)$.

A. **(25 pts)** Implement the Dijkstra algorithm and plan in the generated discrete C-space from the Task 1. On this first iteration, use the heuristic function $h(q, q_G) = 0$, which is equivalent to the Dijkstra algorithm. Report the number of visited states and the final cost. Save the result of the calculated plan in `rod_solve.gif` using `plot_env_3d(environment, rod, plan, save_path='rod_solve.gif')` from `utils.py`, where `plan` is the list of rod states from start to goal.

B. **(10 pts)** Change the heuristic function now to be $h(q, q_G) =$ L1 norm of the x, y components. Comment on the changes: how many states have been visited compared to Dijkstra? What is the final cost? Comment on the results.

C. **(10 pts)** Propose a heuristic function $h(q, q_G)$ that includes orientation. Is it admissible? Justify your answer. Compare all three heuristics of this task in one table (visited states and final cost) and comment on the results. How do they relate to the results of Task 1?